

To connect your React frontend to your backend API, follow this complete, step-by-step implementation.

This guide assumes your backend runs on port `3000` (Express) or `8000` (FastAPI).

Step 1: Fix the CORS Issue on Your Backend

Browsers block React from talking to your API without **CORS** configuration. You must open this gate on your backend first.

If you used Node.js (Express):

1. Stop your backend server. Run this in your backend folder:

```
bash
```

```
npm install cors
```

Use code with caution.

2. Open your backend `server.js` file and add these two lines near the top:

```
javascript
```

```
const cors = require('cors');  
app.use(cors()); // Allows all origins to connect
```

Use code with caution.

3. Restart your backend server (`node server.js`).

If you used Python (FastAPI):

1. Open your backend `main.py` file.
2. Import and add the standard CORS middleware at the top:

```
python
```

```
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], # Allows your React app to connect
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Use code with caution.

3. Restart your backend server (`uvicorn main:app --reload --port 8000`).



Step 2: Create a Fresh React Project

Open a **new, separate terminal window**. Navigate to your workspace folder (outside of your backend folder) and run these commands to generate your frontend project:

bash

```
# 1. Create a clean Vite + React app
npm create vite@latest task-frontend -- --template react

# 2. Move inside your new frontend project folder
cd task-frontend

# 3. Install the standard node dependencies
npm install

# 4. Start the frontend development server
npm run dev
```

Use code with caution.

Keep this terminal running. It will give you a local URL like `http://localhost:5173` to open in your web browser.

Step 3: Write the API Service Layer

We will isolate all network logic inside a single file. If your API address changes, you only have to fix it here.

1. Inside your new frontend project, go to the `src` folder.
2. Create a new folder named `services`.
3. Create a file inside it named `api.js` (`src/services/api.js`).
4. Paste this clean `fetch` wrapper configuration code:

javascript

```
// CHANGE THIS TO 8000 IF YOU ARE USING FASTAPI
const API_BASE_URL = "http://localhost:3000";

export const apiService = {
  // GET /stats
  async getStats() {
    const res = await fetch(`${API_BASE_URL}/stats`);
    if (!res.ok) throw new Error("Failed to fetch stats");
    return res.json();
  },

  // GET /tasks
  async getTasks() {
    const res = await fetch(`${API_BASE_URL}/tasks`);
    if (!res.ok) throw new Error("Failed to fetch tasks");
    return res.json();
  },

  // POST /tasks
  async createTask(title) {
    const res = await fetch(`${API_BASE_URL}/tasks`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ title })
    });
```

```
if (!res.ok) {
  const errData = await res.json().catch(() => ({}));
  throw new Error(errData.error || "Failed to create task");
}
return res.json();
},

// PUT /tasks/:id (Toggle status)
async toggleTaskStatus(id, currentStatus) {
  const res = await fetch(`${API_BASE_URL}/tasks/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ done: !currentStatus })
  });
  if (!res.ok) throw new Error("Failed to update task");
  return res.json();
},

// DELETE /tasks/:id
async deleteTask(id) {
  const res = await fetch(`${API_BASE_URL}/tasks/${id}`, {
    method: "DELETE"
  });
  if (!res.ok) throw new Error("Failed to delete task");
  return true; // 204 No Content has no JSON body
}
};
```

Use code with caution.



Step 4: Assemble the Interface Component

Now, let's link that service file directly to the visual user interface state.

Open your main `src/App.jsx` file, wipe out the default boilerplate code completely, and paste this fully operational implementation:

jsx

```
import { useState, useEffect } from "react";
import { apiService } from "../services/api";

function App() {
  // System State loops
  const [tasks, setTasks] = useState([]);
  const [stats, setStats] = useState({ total: 0, done: 0, open: 0 });
  const [newTitle, setNewTitle] = useState("");
  const [isLoading, setIsLoading] = useState(true);
  const [errorMessage, setErrorMessage] = useState("");

  // Sync data automatically when page loads
  useEffect(() => {
    loadDashboardData();
  }, []);

  const loadDashboardData = async () => {
    setIsLoading(true);
    setErrorMessage("");
    try {
      const [tasksData, statsData] = await Promise.all([
        apiService.getTasks(),
        apiService.getStats()
      ]);
      setTasks(tasksData);
      setStats(statsData);
    } catch (err) {
      setErrorMessage(err.message || "Could not connect to the backend server.");
    } finally {
      setIsLoading(false);
    }
  };

  const handleCreateTask = async (e) => {
    e.preventDefault();
    if (!newTitle.trim()) return;

    try {
      setErrorMessage("");
      await apiService.createTask(newTitle);
      setNewTitle("");
      await loadDashboardData(); // Refresh list and counters
    } catch (err) {
```

```

    setErrorMessage(err.message);
  }
};

const handleToggleDone = async (id, currentStatus) => {
  try {
    await apiService.toggleTaskStatus(id, currentStatus);
    await loadDashboardData();
  } catch (err) {
    alert("Error updating task status");
  }
};

const handleDeleteTask = async (id) => {
  try {
    await apiService.deleteTask(id);
    await loadDashboardData();
  } catch (err) {
    alert("Error deleting task");
  }
};

return (
  <div style={{ maxWidth: "600px", margin: "40px auto", padding: "0 20px", fontFamily: "
    <h1> 📋 Task Dashboard</h1>

    {/* Stats Section */}
    <div style={{ display: "flex", gap: "20px", background: "#f4f4f4", padding: "15px",
      <div><strong>Total:</strong> {stats.total}</div>
      <div><strong>Done:</strong> ✅ {stats.done}</div>
      <div><strong>Open:</strong> ⏳ {stats.open}</div>
    </div>

    {/* Input Form */}
    <form onSubmit={handleCreateTask} style={{ display: "flex", gap: "10px", marginBotto
      <input
        type="text"
        placeholder="What needs to be done?"
        value={newTitle}
        onChange={(e) => setNewTitle(e.target.value)}
        style={{ flex: 1, padding: "10px", borderRadius: "4px", border: "1px solid #ccc"
      />
      <button type="submit" style={{ padding: "10px 20px", background: "#0070f3", color:
        Add Task

```

```

    </button>
  </form>

  {/* Notification Errors */}
  {errorMessage && (
    <div style={{ color: "red", background: "#ffebee", padding: "10px", borderRadius:
      ⚠️ {errorMessage}
    </div>
  )}

  {/* Task Collection List */}
  {isLoading ? (
    <p>Loading database items...</p>
  ) : tasks.length === 0 ? (
    <p>No tasks found. Relax or add a new one!</p>
  ) : (
    <ul style={{ listStyle: "none", padding: 0 }}>
      {tasks.map((task) => (
        <li key={task.id} style={{ display: "flex", alignItems: "center", justifyConte
          <div style={{ display: "flex", alignItems: "center", gap: "10px" }}>
            <input
              type="checkbox"
              checked={task.done}
              onChange={() => handleToggleDone(task.id, task.done)}
              style={{ width: "18px", height: "18px", cursor: "pointer" }}
            />
            <span style={{ textDecoration: task.done ? "line-through" : "none", color:
              {task.title}
            </span>
          </div>
          <button onClick={() => handleDeleteTask(task.id)} style={{ background: "#ff4
            Delete
          </button>
        </li>
      )})
    </ul>
  )}
</div>
);
}

export default App;

```

Use code with caution.

Step 5: Commit Your Code

Now save your frontend progress via git commands:

```
bash
```

```
git add .
```

```
git commit -m "Integration: React app connected to full CRUD backend API"
```